Docker Swarm 用法

大綱

1. 基本概念
2. 叢集架構
3. 初始化 Swarm
4. 節點管理
5. 服務部署
6. 服務管理
7. 滾動更新
8. 叢集網路
9. 負載平衡
10. 儲存管理
11. 實際範例

---

1. 基本概念

1.1 什麼是 Docker Swarm?

Docker Swarm 是 Docker 內建的容器編排工具，用於管理多台 Docker 主機，形成叢集（Cluster），並協調容器的高可用性部署。

1.2 Swarm 特色

- **原生整合**：無需額外安裝，已內建於 Docker Engine
- **分散式協調**：使用 Raft 共識演算法
- **滾動更新**：支援零停機部署更新
- **服務發現**：內建 DNS 自動服務發現
- **負載平衡**：內建負載平衡機制
- **高可用性**：支援多重管理節點（Manager）

1.3 Swarm vs Kubernetes

功能	Docker Swarm	Kubernetes
安裝難度	簡單	複雜
學習曲線	較平緩	較陡峭
雲端支援	需自行架設	多雲支援佳
擴展性	中等	極佳
生態系統	中等	龐大

1.4 適用場景

- 中小規模叢集（數十節點）
- 快速部署與原型開發
- 已有 Docker 技術棧的團隊
- 不需要複雜的編排功能

---

2. 叢集架構

2.1 節點類型

```
┌──────────────────────────────────────────┐
│              Swarm Cluster               │
│                                          │
│   ┌──────────────────────────────────┐   │
│   │      Manager Node (Leader)       │   │
│   │   - 排程任務                      │   │
│   │   - 維護叢集狀態                  │   │
│   │   - 處理 API 請求                 │   │
│   └──────────────────────────────────┘   │
│          ▲              ▲                │
│          │              │                │
│   ┌──────┴──────┐  ┌──────┴──────┐       │
│   │   Manager   │  │   Manager   │       │
│   │    Node     │  │    Node     │       │
│   └─────────────┘  └─────────────┘       │
│          ▲              ▲                │
│          │              │                │
│   ┌──────┴──────┐  ┌──────┴──────┐       │
│   │   Worker    │  │   Worker    │       │
│   │    Node     │  │    Node     │       │
│   └─────────────┘  └─────────────┘       │
│                                          │
│   ┌─────────────┐  ┌─────────────┐       │
│   │   Worker    │  │   Worker    │       │
│   │    Node     │  │    Node     │       │
│   └─────────────┘  └─────────────┘       │
└──────────────────────────────────────────┘
```

2.2 Manager 節點職責

- 維護叢集狀態（key-value store）
- 排程任務到 Worker 節點
- 處理管理者 API 請求
- 執行共識演算法（Raft）確保一致性

2.3 Worker 節點職責

- 執行容器任務
- 回報容器狀態
- 從 Manager 接收指令

2.4 節點數量建議

叢集規模	Manager 數量	建議
1-3 節點	1	測試環境
3-5 節點	3	小型生產環境
5-10 節點	5	中型生產環境
10+ 節點	5-7	大型生產環境

---

3. 初始化 Swarm

3.1 初始化 Swarm（單一節點）

```
# 初始化 Swarm,, 成為 Manager
docker swarm init

# 輸出範例:
# Swarm initialized: current node (xxxxx) is now a manager.
# To add a worker to this swarm, run the following command:
# docker swarm join --token SWMTKN-xxxxx 192.168.1.100:2377
```

3.2 初始化 Swarm（高可用模式）

```
# 在第一個 Manager 上初始化，指定advertise位址
docker swarm init --advertise-addr 192.168.1.100

# 查看現有 token
docker swarm join-token worker
docker swarm join-token manager
```

3.3 加入 Worker 到叢集

在其他主機上執行:

```
# 使用 Worker token 加入
docker swarm join --token SWMTKN-xxxxx-xxxxx 192.168.1.100:2377

# 驗證加入成功
docker node ls
```

3.4 加入 Manager 到叢集

```
# 使用 Manager token 加入
docker swarm join --token SWMTKN-xxxxx-xxxxx 192.168.1.101:2377

# 如果是新 Manager，需要新增為 Manager
docker node promote <node-id>
```

3.5 離開 Swarm

```
# Worker 節點離開
docker swarm leave

# Manager 節點離開（需先移除）
docker swarm leave --force
```

---

4. 節點管理

4.1 查看節點列表

```
# 查看所有節點
docker node ls

# 輸出範例:
# ID                            HOSTNAME   STATUS    AVAILABILITY   MANAGER STATUS
# xxxxx_worker1    worker1      Ready       Active
# xxxxx_worker2    worker2      Ready       Active
# xxxxx_manager1   manager1     Ready       Active    Leader
# xxxxx_manager2   manager2     Ready       Active    Reachable
```

4.2 節點狀態

欄位	說明
STATUS	Ready（正常）/ Down（故障）
AVAILABILITY	Active（可排程）/ Pause（暫停）/ Drain（排水）
MANAGER STATUS	Leader / Reachable / Unreachable / 空（Worker）

4.3 節點維護

```
# 將節點設為排水模式（不再分配新任務）
docker node update --availability drain worker1

# 將節點設為暫停模式（維持當前任務，不分配新任務）
```

```
docker node update --availability pause worker1

# 恢復為正常
docker node update --availability active worker1

# 移除 Worker 節點
docker node rm worker1

# 新增節點標籤
docker node update --label-add region=us-west worker1

# 查看節點詳細資訊
docker node inspect worker1
```

---

5. 服務部署

5.1 建立服務

```
# 基本建立服務
docker service create --name web nginx:latest

# 指定副本數
docker service create --name web --replicas 3 nginx:latest

# 指定容器連接埠
docker service create --name web -p 80:80 --replicas 3 nginx:latest

# 指定環境變數
docker service create --name app --env NODE_ENV=production my-app:latest

# 掛載磁碟區
docker service create --name web --mount type=volume,source=web-data,target=/data
nginx:latest
```

5.2 使用 Compose 檔案部署

建立 `docker-stack.yml`：

```
version: "3.8"

services:
  web:
    image: nginx:latest
    ports:
      - "80:80"
    replicas: 3
    placement:
      constraints:
```

```
          - node.role == worker
      update_config:
        parallelism: 1
        delay: 10s
      restart_policy:
        condition: on-failure

    app:
      image: my-app:latest
      replicas: 2
      environment:
        - DB_HOST=db
      depends_on:
        - db

    db:
      image: mysql:8.0
      placement:
        constraints:
          - node.role == manager
      volumes:
        - db-data:/var/lib/mysql

  volumes:
    db-data:

  networks:
    app-net:
      driver: overlay
```

部署：

```
# 部署 Stack
docker stack deploy -c docker-stack.yml myapp

# 查看 Stack 列表
docker stack ls

# 查看 Stack 服務
docker stack ps myapp

# 移除 Stack
docker stack rm myapp
```

5.3 部署模式

全域模式（Global）

每個節點都會執行一個容器：

```
docker service create --mode global nginx:latest
```

複本模式（Replicated）

在指定數量的節點上執行容器（預設）：

```
docker service create --replicas 5 nginx:latest
```

---

6. 服務管理

6.1 查看服務

```
# 查看所有服務
docker service ls

# 查看服務詳細資訊
docker service inspect web

# 查看服務副本狀態
docker service ps web

# 查看服務日誌
docker service logs web -f
```

6.2 調整服務規模

```
# 擴展副本數
docker service scale web=5

# 同時調整多個服務
docker service scale web=5 app=3

# 更新副本數
docker service update --replicas 3 web
```

6.3 更新服務

```
# 更新映像檔版本
docker service update --image nginx:1.25 web

# 更新環境變數
docker service update --env-add NODE_ENV=production app
```

```
# 更新連接埠
docker service update --publish-rm 80:80 --publish-add 8080:80 web

# 滾動重啟
docker service update --force web
```

6.4 移除服務

```
# 移除服務
docker service rm web

# 一次移除多個
docker service rm web app db
```

---

7. 滾動更新

7.1 更新設定

```
services:
  web:
    image: nginx:latest
    update_config:
      parallelism: 1        # 每次更新容器數
      delay: 10s             # 每批間隔時間
      failure_action: rollback  # 失敗時回滾
      monitor: 10s           # 監控時間
      max_failure_ratio: 0.3 # 最大失敗率
      order: start-first     # 更新順序: start-first / stop-first
    rollback_config:
      parallelism: 1
      delay: 5s
      failure_action: pause
```

7.2 手動滾動更新

```
# 手動更新服務映像檔
docker service update --image nginx:1.25 web

# 查看更新進度
watch docker service ps web

# 發生問題時手動回滾
docker service rollback web
```

7.3 健康檢查

```yaml
services:
  web:
    image: nginx:latest
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 40s
```

---

8. 叢集網路

8.1 網路驅動程式

驅動程式	說明
`overlay`	跨主機容器通訊（Swarm 專用）
`ingress`	Swarm 負載平衡網路（內部使用）
`docker_gwbridge`	橋接網路（主機間通訊）

8.2 建立 overlay 網路

```bash
# 建立覆寫網路（支援加密）
docker network create -d overlay my-overlay

# 建立加密覆寫網路
docker network create -d overlay --encrypt my-overlay-encrypted
```

8.3 使用網路

```yaml
services:
  web:
    networks:
      - app-net
  app:
    networks:
      - app-net

networks:
  app-net:
    driver: overlay
```

---

9. 負載平衡

9.1 內建負載平衡

Docker Swarm 內建兩層負載平衡：

1. **入口層（Ingress）**：將外部流量分散到服務
2. **服務層（VIP）**：將流量分散到容器副本

9.2 DNS 輪詢

```
# 服務名稱自動註冊 DNS
# web 服務可通過 web.myapp 訪問
```

9.3 對外暴露服務

```
# 指定模式暴露
docker service create --name web --publish mode=ingress,target=80,published=80
nginx:latest

# 長期發布模式
docker service create --name web --publish mode=replicated,target=80,published=80
--publish-add mode=replicated,target=443,published=443 nginx:latest
```

---

10. 儲存管理

10.1 本機儲存（local）

```
volumes:
  web-data:
    driver: local

services:
  web:
    volumes:
      - web-data:/app/data
```

10.2 配置檔案

```
# 建立配置
echo "config content" | docker config create web-config -
```

```
# 使用配置
docker service create --config web-config --name web nginx:latest
```

```
services:
  web:
    configs:
      - source: web-config
        target: /etc/nginx/nginx.conf

configs:
  web-config:
    file: ./nginx.conf
```

10.3 密碼

```
# 建立密碼
echo "supersecret" | docker secret create db-password -

# 使用密碼
docker service create --secret db-password --name db mysql:8.0
```

```
services:
  db:
    secrets:
      - db-password

secrets:
  db-password:
    file: ./db-password.txt
```

---

11. 實際範例

11.1 完整 Web 應用程式部署

```
version: "3.8"

services:
  nginx:
    image: nginx:alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
```

```yaml
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    networks:
      - app-net
    placement:
      constraints:
        - node.role == worker
    replicas: 2
    update_config:
      parallelism: 1
      delay: 10s
    restart_policy:
      condition: on-failure

  api:
    image: my-api:latest
    environment:
      - DB_HOST=db
      - REDIS_HOST=redis
    networks:
      - app-net
    deploy:
      replicas: 3
      update_config:
        parallelism: 1
        delay: 5s
      restart_policy:
        condition: on-failure

  worker:
    image: my-worker:latest
    networks:
      - app-net
    deploy:
      replicas: 2

  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
      MYSQL_DATABASE: app
    volumes:
      - db-data:/var/lib/mysql
    networks:
      - app-net
    placement:
      constraints:
        - node.role == manager

  redis:
    image: redis:7-alpine
    volumes:
      - redis-data:/data
    networks:
      - app-net
```

```
volumes:
  db-data:
  redis-data:

networks:
  app-net:
    driver: overlay
```

部署：

```
# 建立 .env 檔案
echo "DB_ROOT_PASSWORD=secretpassword" > .env

# 部署 Stack
docker stack deploy -c docker-stack.yml myapp

# 監控服務狀態
docker service ps myapp_nginx

# 水平擴展
docker service scale myapp_api=5

# 更新服務
docker service update --image my-api:v2 myapp_api

# 滾動回滾
docker service rollback myapp_api

# 移除整個 Stack
docker stack rm myapp
```

總結

- **初始化**：`docker swarm init` 啟動叢集
- **加入節點**：使用 `join-token` 讓其他主機加入
- **部署服務**：`docker service create` 或 Stack 檔案
- **規模調整**：`docker service scale`
- **滾動更新**：`update_config` 自動更新
- **網路**：使用 overlay 驅動程式實現跨主機通訊
- **儲存**：使用 local volume、config、secret 管理資料與配置